

FinanceHub Pro

Plataforma de Gestão Financeira Corporativa

Documento de Requisitos Técnicos v2.1

Campo	Valor
Empresa	Agilize TI Ltda
Responsável	Carlos Eduardo Mendes — Gerente de Projetos
Data	2026-04-09
Versão	2.1
Classificação	Confidencial
Criticidade	Alta

1. Visão Geral do Projeto

O **FinanceHub Pro** é uma plataforma web de gestão financeira corporativa que consolida fluxo de caixa, contas a pagar/receber, conciliação bancária, emissão de notas fiscais e relatórios gerenciais em um único sistema. O projeto visa substituir planilhas e sistemas legados fragmentados por uma solução integrada, segura e auditável.

Nome do projeto: FinanceHub Pro

Slug: financehub-pro

Projeto existente? Não — sistema novo

Tipo de iniciativa: Novo sistema

Criticidade: Alta — operação financeira real

Classificação da informação: Confidencial

2. Perfil de Entrega e Arquitetura

2.1 Entregáveis

- Web application (SPA)
- REST API
- Dashboard operacional
- Workers para processamento assíncrono

2.2 Arquitetura

Monólito modular com separação clara de domínios (DDD). Cada módulo financeiro (Caixa, NF-e, Conciliação, Relatórios) é um bounded context independente, comunicando via eventos de domínio internos. Preparado para futura extração em microserviços.

- Monólito modular (fase 1)
- Event-driven interno
- Containerizado via Docker
- Cloud (AWS)
- Alta disponibilidade: Sim
- Multi-tenant: Sim — isolamento por schema PostgreSQL
- Processamento assíncrono: Sim — filas para NF-e e conciliação bancária

3. Frontend (Interface do Usuário)

Tem frontend? Sim

Aspecto	Especificação
Tipo	Web SPA + Painel admin + Portal autenticado
Stack	React com Vite
Linguagem	TypeScript
Responsividade	Sim — tablets e desktops (min 768px)

Aspecto	Especificação
Acessibilidade	WCAG 2.1 AA
Dark theme	Sim
Formulários complexos	Sim — wizard de emissão NF-e com 4 etapas
Gráficos	Sim — dashboards financeiros (Recharts)
Upload de arquivos	Sim — extratos bancários (OFX, CSV)
Geração PDF	Sim — relatórios e boletos
i18n	Sim — pt-BR e en-US

4. Backend e APIs

Tem backend? Sim

Aspecto	Especificação
Linguagem	Python 3.12
Framework	FastAPI
Tipo	REST API + WebSocket (notificações real-time) + Workers (Celery)
Autenticação	JWT com refresh token + OAuth2 (Google SSO)
RBAC	Sim — Admin, Gerente Financeiro, Analista, Auditor, Leitura
Webhooks	Sim — notificações de pagamento e conciliação
Jobs agendados	Sim — conciliação diária, relatórios semanais
Auditoria	Sim — log de todas as operações financeiras
Versionamento API	Sim — /api/v1/, /api/v2/
Rate limiting	Sim — 100 req/min por tenant
Observabilidade	Sim — OpenTelemetry + Prometheus
Integração IA	Sim — classificação automática de lançamentos

5. Dados, Cache e Mensageria

Componente	Escolha	Finalidade
Banco principal	PostgreSQL 16	Transacional + analítico (partições por mês)
Perfil de uso	Transacional + Analítico	OLTP diário + relatórios gerenciais
Redis	Sim	Cache de leitura, sessões JWT, rate limiting, locks distribuídos
Mensageria	Sim (RabbitMQ)	Eventos de domínio, integrações async, background jobs
n8n	Sim	Automação de integrações bancárias, notificações, relatórios

6. IA, Segurança e Observabilidade

6.1 Inteligência Artificial

Usa IA? Sim

- Provedor: Anthropic (Claude) — classificação de lançamentos e análise de padrões
- Finalidade: Classificação automática, análise de requisitos, documentação técnica
- Restrições: Mascaramento de dados financeiros sensíveis + anonimização de PII
- Avaliação por tipo de dado antes de envio ao modelo

6.2 Segurança

- JWT com rotação automática a cada 15 min
- OAuth2 (Google SSO) para login corporativo
- MFA obrigatório para operações financeiras acima de R\$ 10.000
- Criptografia em trânsito (TLS 1.3) e em repouso (AES-256)
- Vault (HashiCorp) para gestão de credenciais
- Rotação automática de credenciais a cada 90 dias
- Trilhas de auditoria completas — imutáveis, com retenção de 7 anos

6.3 Observabilidade

- Logs estruturados (JSON) via structlog
- Métricas de negócio e infra (Prometheus + Grafana)
- Tracing distribuído (OpenTelemetry → Jaeger)
- Health checks em todos os serviços
- Alertas automáticos (PagerDuty)
- Dashboard operacional (Grafana)
- Dashboard executivo (Metabase)

7. Testes, Validação e Entregáveis

7.1 Estratégia de Testes

Tipo	Ferramenta	Cobertura alvo
Unitários	pytest	≥ 80% por módulo

Tipo	Ferramenta	Cobertura alvo
Integração	pytest + testcontainers	Todos os endpoints
Contrato/API	schemathesis	OpenAPI spec compliance
E2E	Playwright	Fluxos críticos (pagamento, NF-e)
Regressão	pytest (suite completa)	Pré-release
Segurança (SAST)	Bandit + Semgrep	CI pipeline
Performance	Locust	100 users simultâneos
Smoke	curl + health checks	Deploy pipeline
UAT	Manual + checklist	Pré-produção

Quality gate automatizado? Sim — CI bloqueia merge se cobertura < 80% ou SAST detectar vulnerabilidade

Exige evidência formal QA? Sim — relatórios de teste anexados ao release

7.2 Entregáveis Esperados

- Sugestão de arquitetura validada
- Sugestão de stack com justificativa
- Documento técnico completo
- Documento de negócio (requisitos funcionais)
- Gap analysis (lacunas identificadas)
- Backlog priorizado (MVP → v2 → v3)
- Plano de testes detalhado
- Plano de segurança
- Plano de observabilidade
- Plano de deploy (CI/CD pipeline)

7.3 Formato de Retorno

Painel GCA, PDF, Markdown, JSON